



UMCS

UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Mateusz Kowal

nr albumu: 318683

Środowisko programowalnego bezzałogowego statku powietrznego z użyciem mikrokomputera Raspberry Pi

A programmable unmanned aerial vehicle environment
based on a Raspberry Pi microcomputer

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. profesor uczelni Beaty Byliny**

Lublin 2026

Spis treści

Wstęp	5
1 Wprowadzenie do bezzałogowych statków powietrznych	7
1.1 Podstawowe pojęcia	7
1.2 Zadania i ograniczenia komputera pokładowego	7
1.2.1 Podstawowe zadania komputera pokładowego	7
1.2.2 Opcjonalne zastosowania autopilota	8
1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym	8
2 Szczegóły techniczne zbudowanego środowiska	11
2.1 Lista komponentów i ich specyfikacja	11
2.1.1 Rama BSP	11
2.1.2 Napęd	11
2.1.3 Zewnętrzny moduł GNSS	13
2.1.4 Układ komunikacji z pilotem	13
2.1.5 Komputer pokładowy	13
2.1.6 Komputer asystujący	14
2.1.7 Źródło zasilania	14
2.2 Specyfikacja drona	14
3 Zastosowanie środowiska	17
3.1 Wstępne założenia	17
3.2 Przykład użycia	20
3.3 Kod programu	21
3.4 Testy	24
3.4.1 Sprawdzenie poprawności działania systemu	24
3.4.2 Pobór prądu	24
3.4.3 Dokładność pomiarów odległości	25
3.4.4 Czas reakcji systemu	25
3.5 Alternatywne rozwiązania	25

4	Potencjalne ścieżki rozwoju projektu	27
4.1	Półautonomiczny system zapobiegania kolizjom	27
4.2	Wielokierunkowy monitoring	27
4.3	Sieć neuronowa rozpoznawanie obiektów z obrazu kamery	27
4.4	mapa 3d lidar	27
	Bibliografia	27

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Wprowadzenie do bezzałogowych statków powietrznych

1.1 Podstawowe pojęcia

Bezzałogowy statek powietrzny, skrót BSP, to kategoria pojazdów poruszających się w powietrzu do której zaliczane są samoloty, śmigłowce i pojazdy wielowirnikowe (w tym drony), które odbywają loty bez pilota na pokładzie i bez możliwości zabierania pasażerów. Sterowanie może się odbywać zdalnie przez pilota naziemnego lub przez komputer, jeśli pojazd jest autonomiczny [2].

Komputer pokładowy jest urządzeniem niezbędnym do działania BSP. Jego głównym zadaniem jest sterowanie silnikami pojazdu oraz otrzymywanie i przetwarzanie sygnałów docierających z aparatury sterującej. W tym przypadku rolę komputera pokładowego pełni Pixhawk 6C.

Komputer asystujący to komputer znajdujący się na pokładzie BSP, połączony z komputerem pokładowym. Jego rolą jest wykonywanie zadań zbyt skomplikowanych obliczeniowo by móc powierzyć je komputerowi pokładowemu lub wymagają elementów przez komputer pokładowy nieposiadanych i niewspieranych. Komputerem asystującym w przypadku tego projektu jest Raspberry Pi 5B.

1.2 Zadania i ograniczenia komputera pokładowego

1.2.1 Podstawowe zadania komputera pokładowego

Rolą każdego komputera pokładowego jest przede wszystkim sterowanie silnikami pojazdu którego jest częścią. Komputer pokładowy musi więc posiadać odpowiednie interfejsy do komunikacji z silnikami oraz interfejs do otrzymywania poleceń. W przy-

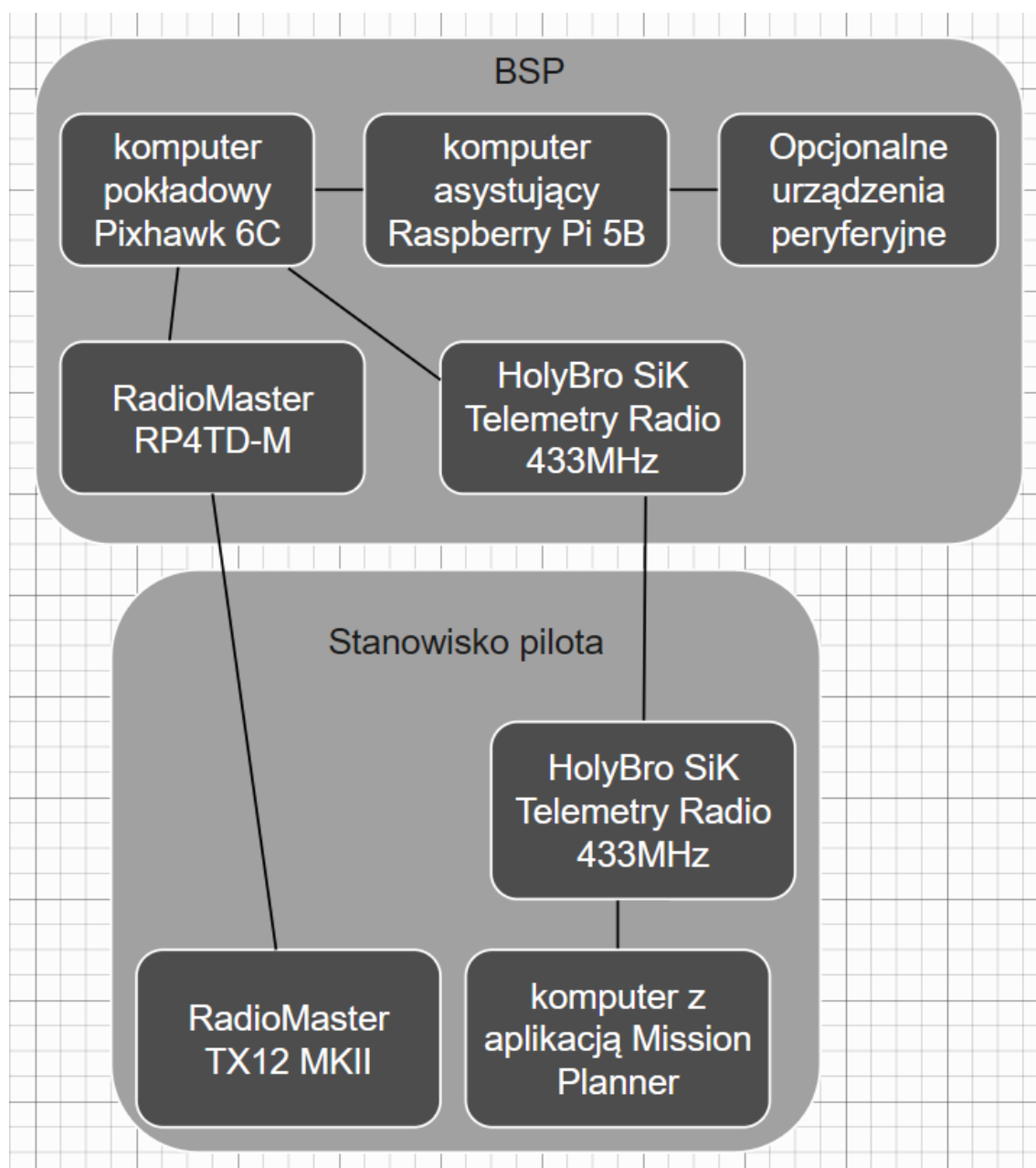
padku komputerów pokładowych używanych w BSP wielowirnikowych niezbędny jest również żyroskop, dzięki któremu komputer pokładowy zna swoją pozycję i jest w stanie utrzymać pojazd poziomo lub odpowiednio odchylić go zgodnie z poleceniami które otrzyma.

1.2.2 Opcjonalne zastosowania autopilota

W zależności od modelu komputera pokładowego oraz zainstalowanego na nim oprogramowania może on dodatkowo obsługiwać wiele urządzeń peryferyjnych i posługiwać się bardziej złożoną logiką. Przeciętny komputer pokładowy dostępny na rynku cywilnym oferuje możliwość podłączenia zewnętrznego modułu GPS czy kamery, a także samodzielnie przechowywać i wykonywać polecenia pozwalające na lot autonomiczny. Standardem jest również możliwość ustawienia procedury bezpieczeństwa (*ang. failsafe*), która decyduje jak zachowa się pojazd w razie utraty podłączenia z kontrolerem naziemnym lub jakimkolwiek innym urządzeniem wydającym polecenia komputerowi pokładowemu.

1.3 Rozszerzenie możliwości BSP dzięki komputerom asystującym

Jeśli użytkownik chce wykorzystać swój BSP w bardziej nietypowy sposób, na drodze mogą stanąć mu ograniczenia sprzętowe. Komputery pokładowe oferują zazwyczaj niewielką moc obliczeniową i ograniczoną ilość i różnorodność gniazd do podłączenia urządzeń peryferyjnych. Jednym ze sposobów przystosowania BSP do naszych wymagań jest użycie komputera asystującego który rozszerzy możliwości jednostki. Przykładowo, użytkownik może chcieć użyć swojego BSP do monitorowania dużego obszaru. W tym celu chciałby zamontować kamery z wielu stron, jednak użyty przez niego komputer pokładowy ma możliwość podpięcia tylko jednej kamery. Użycie komputera asystującego może być wtedy rozwiązaniem problemu. W zależności od użytego komputera asystującego można użyć nawet tak złożonych obliczeniowo programów jak sztuczna sieć neuronowa analizująca dane z urządzeń wejściowych i wydająca polecenia jednostce. Sposób połączenia komputera asystującego oraz połączeń dających użytkownikowi kontrolę nad dronem i informacje zwrotne zostały przedstawione na rysunku [1.1](#).



Rysunek 1.1: Schemat wybranych komponentów jednostki i połączenia między nimi

Rozdział 2

Szczegóły techniczne zbudowanego środowiska

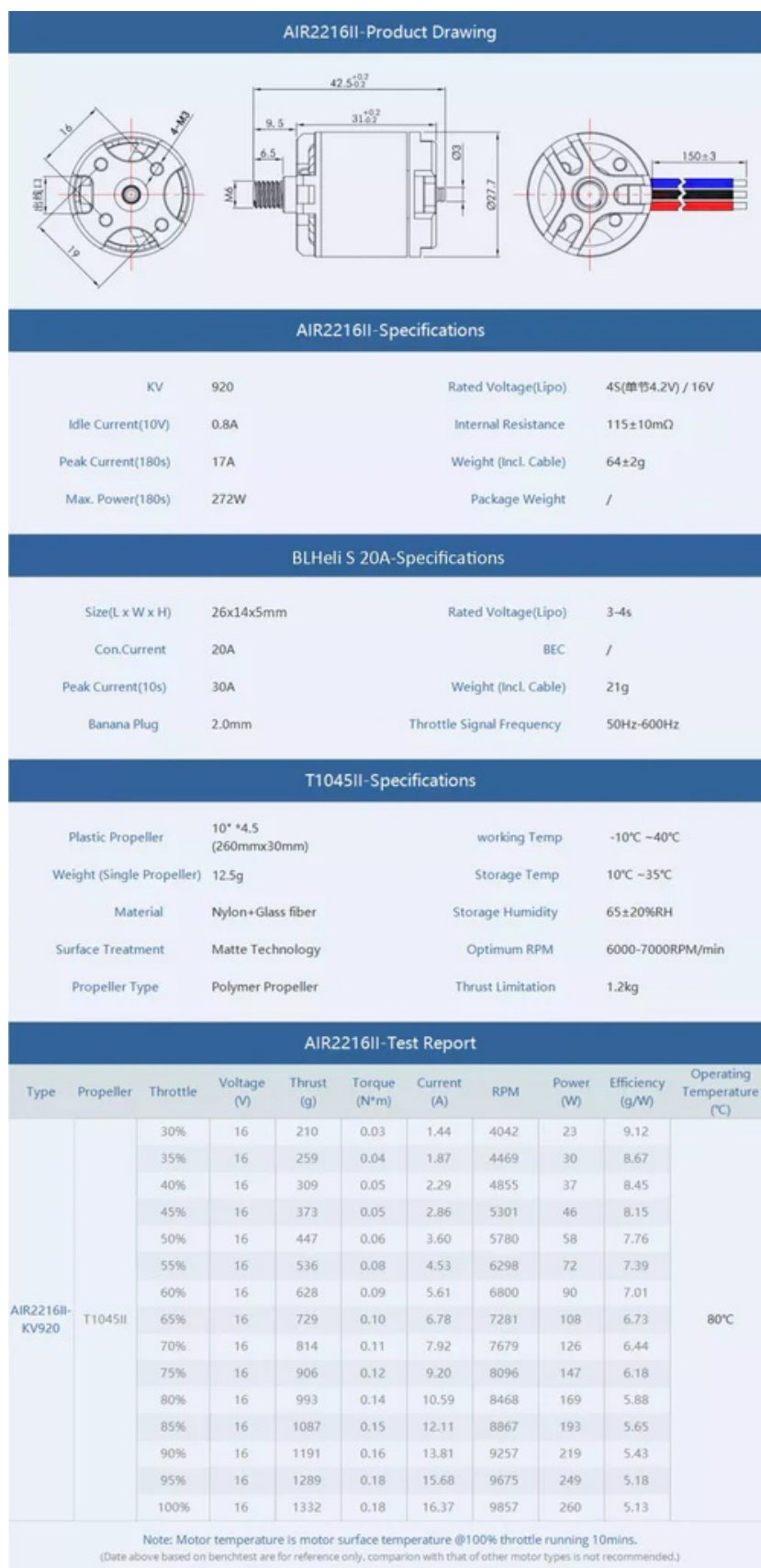
2.1 Lista komponentów i ich specyfikacja

2.1.1 Rama BSP

Rama użyta do zbudowania jednostki to Holybro S500 v2. Oferuje wbudowaną w konstrukcję kadłuba rozdzielnicę napięcia (*ang. power distribution board*) dostarczającą energię elektryczną z akumulatora do silników. Miejscem na montaż silników są cztery ramiona wykonane z nylonu poliamidowego, wzmocnione w środku poprzez dodanie prętów z włókna węglowego. Całość opiera się na podwoziu z rurek z włókna węglowego połączonych plastikowymi łącznikami.

2.1.2 Napęd

Pojazd napędzany jest przez cztery silniki bezszczotkowe AIR2216II osiągające do 272W. Silniki bezszczotkowe wymagają zasilania prądem zmiennym trójfazowym, dlatego do ich działania potrzebne są moduły elektronicznej kontroli prędkości (*ang. electronic speed controller*). Moduły te otrzymują zasilanie prądem stałym z akumulatora oraz sygnał z komputera pokładowego sterujący prędkością obracania się silników. Następnie generuje prąd zmienny trójfazowy który wywołuje obrót silników. Moduły ESC zastosowane w zbudowanym pojeździe to BLHeli S 20A. Jednostka została wyposażona w śmigła T1045II o rozpiętości 260 milimetrów i szerokości do 30 milimetrów. Specyfikacja każdego z elementów napędu została pokazana na rysunku [2.1](#).



Rysunek 2.1: Dokumentacja techniczna AIR2216II, BLHeli S 20A, T1045II. Źródło: [3]

2.1.3 Zewnętrzny moduł GNSS

Zastosowany w jednostce zewnętrzny moduł GNSS to M10 od firmy Holybro. Oferuje jednocześnie otrzymywanie informacji od czterech największych systemów satelit geolokacyjnych: europejskich satelit Galileo, amerykańskich GPS, rosyjskich GLO-NASS i chińskich Bei Dou. Oferowana dokładność wyznaczonej pozycji to 2.0m CEP, co oznacza że co najmniej 50% pomiarów wskaże pozycję oddaloną o 2 metry lub mniej od rzeczywistej pozycji [4].

2.1.4 Układ komunikacji z pilotem

BSP został przygotowany do utrzymywania dwóch rodzajów komunikacji z pilotem. Podstawowym kanałem komunikacji jest kontroler RadioMaster TX12 MKII pozwalający na zdalne sterowanie dronem wyposażonym w odbiornik radiowy RadioMaster RP4TD-M używając systemu ExpressLRS zapewniającego niezawodność i stabilność łącza nawet na dużych dystansach. Drugim kanałem komunikacji jest system telemetry Holybro SiK Telemetry Radio V3 w wersji 433MHz. Moduł podłączony do komputera osobistego znajdującego się na ziemi w pobliżu pilota pozwala na użycie aplikacji Mission Planner— oprogramowania pozwalającego między innymi na śledzenie BSP na mapie czy odczyt bieżących danych [1].

2.1.5 Komputer pokładowy

Rolę komputera pokładowego pełni Pixhawk 6C. Wyposażony jest w dwa procesory: STM32H743 — 480MHz Cortex-M7 odpowiedzialny za działanie oprogramowania zainstalowanego na urządzeniu, odczyt parametrów z sensorów, przetwarzanie informacji i przekazywanie poleceń dotyczących silników do drugiego procesora, w tym przypadku STM32F103 — 72 MHz Cortex-M3. Zadaniem drugiego procesora jest generowanie sygnałów PWM na podstawie otrzymanych danych i przekazywanie ich do modułów ESC. W przypadku wystąpienia problemów z pierwszym procesorem jest w stanie samodzielnie generować sygnał sterujący silnikami, jednak przez brak dostępu do czujników takich jak żyroskopy czy akcelerometry jego możliwości ograniczają się do utrzymania kręcenia się śmigieł w celu spowolnienia upadku i minimalizacji uszkodzeń przy lądowaniu awaryjnym.

Do poprawnego sterowania pojazdem wymagane są również odpowiednie czujniki. Pixhawk 6C posiada dwa urządzenia łączące w sobie akcelerometry i żyroskopy: ICM-42688-P i BMI055. Użycie dwóch urządzeń i porównanie ich odczytów ze sobą pozwala na wykrycie niedokładności lub uszkodzenia jednego z nich i zapobiega destabilizacji drona w przypadku awarii. W razie uszkodzenia komputer pokładowy

może wdrożyć procedurę bezpieczeństwa i bezpiecznie ją wykonać polegając na pozostałym działającym czujniku. Pixhawk 6C wyposażony jest również w magnetometr IST8310 pozwalający na określenie jego pozycji na podstawie pola magnetycznego Ziemi oraz barometr MS5611 pozwalający obliczyć wysokość nad ziemią na podstawie różnicy ciśnień [8]. Oprogramowaniem działającym na komputerze pokładowym jest ArduCopter w wersji 4.6.2.

2.1.6 Komputer asystujący

Asystentem dla Pixhawk 6C jest mikrokomputer Raspberry Pi 5B. Posiada on czterordzeniowy procesor Arm Cortex-A76, 8GB pamięci RAM oraz szereg sposobów na podłączenie urządzeń peryferyjnych, np. cztery gniazda USB czy 40 pinów GPIO. Zainstalowany na nim system operacyjny to Raspberry Pi OS będący portem Debiana Trixie.

2.1.7 Źródło zasilania

BSP zasilany jest za pomocą akumulatora litowo-jonowego Gens Ace G-Tech o pojemności 5000 mAh, napięciu 14.8V i wadze 439g. Za zasilanie komputera asystującego odpowiedzialny jest powerbank z baterią litowo-polimerową o pojemności 10000 mAh, napięciu 5V i mocy maksymalnej 22,5W, wadze 235g. Możliwe jest zasilanie Raspberry Pi akumulatorem drona, wiąże się to jednak z ingerencją w gotowy układ zasilania oraz koniecznością obniżenia napięcia z 14.8V do 5V. Problem stanowią również silniki elektryczne napędzające jednostkę, ich zużycie prądu zmienia się bardzo dynamicznie co stwarza ryzyko spadku napięcia zasilającego komputer asystujący poniżej wymaganych 5V. W przypadku Raspberry Pi spadek napięcia poniżej 4,63V ($\pm 5\%$) może skutkować obniżoną wydajnością komputera, nieprawidłowym działaniem podłączonych urządzeń peryferyjnych lub do nieprzewidywalnego zachowania urządzenia czy uszkodzenia danych na karcie pamięci. [6] Z uwagi na potencjalne problemy i konieczność ingerencji w gotowy układ zasilania zastosowane zostało oddzielne źródło prądu dla komputera asystującego.

2.2 Specyfikacja drona

Waga jednostki, wliczając w to komputer asystujący i jego źródło zasilania, wynosi 1715g. Na podstawie wagi, specyfikacji silników [3] oraz akumulatora można oszacować średni możliwy czas lotu. Aby dron zawisł w bezruchu, każdy z czterech silników musi wytworzyć ok. 428,75g ciągu. Dzięki dokumentacji wiadomo że jeden silnik z zastoso-

wanym dołączonym w zestawie śmigłem potrzebuje 2,86A do wytworzenia 373g ciągu i 3,60A dla 447g ciągu. Prąd potrzebny do wytworzenia dokładnie 428,75g ciągu może być obliczony z użyciem interpolacji liniowej:

$$I = I_1 + \frac{T - T_1}{T_2 - T_1}(I_2 - I_1)$$

$$I = 2.86 + \frac{428.75 - 373}{447 - 373}(3.60 - 2.86)$$

$$I \approx 3.42 \text{ A}$$

Jeden silnik potrzebuje około 3,42A, co dla czterech silników daje łącznie 13,67A. Używając akumulatora o pojemności 5000 mAh daje to teoretyczne 21,95 minut lotu. Biorąc pod uwagę dodatkowe zużycie prądu przez komputer pokładowy oraz zwiększone zużycie prądu przy wznoszeniu i manewrowaniu, bezpieczniejszym będzie przyjęcie marginesu błędu i określenie czasu lotu na około 19–20 minut. Oszacowanie maksymalnego czasu działania Raspberry Pi jest dużo trudniejsze, zależy to w dużym stopniu od złożoności programów na niej uruchomionych. Artykuł ogłaszający wypuszczenie na rynek Raspberry Pi 5 w wersji z 16GB pamięci RAM określa zużycie w spoczynku na 2–3W [5], zakładając zużycie 3W przy 10000mAh i 5V daje to około 16,7 godzin działania. Trzeba jednak wziąć pod uwagę że to ogłoszenie nie jest tak wiarygodnym źródłem jak oficjalna dokumentacja, która w tym przypadku nie wspomina o poborze mocy w stanie spoczynku. Pobór prądu w praktyce będzie też większy ze względu na uruchomione na mikrokomputerze programy. Opisane dalej testy wykazały że pobór mocy w stanie spoczynku wahają się między 3.177W a 3.484W, większość odczytów była zbliżona do 3,281W. Przy parametrach zastosowanego zasilania szacowany czas działania to 11 godzin i 15 minut.

Rozdział 3

Zastosowanie środowiska

3.1 Wstępne założenia

Komputer Raspberry Pi jest połączony z Pixhawk 6C za pomocą USB. W katalogu `~/uavproject` znajduje się wirtualne środowisko python oraz wszystkie programy współpracujące z dronem. Komunikacja z dronem możliwa jest dzięki bibliotekom DroneKit 2.9.2 i PyMAVLink 2.4.49. Użycie tych bibliotek ogranicza wersję pythona do nie nowszej niż 3.9.18, ponieważ od jakiegoś czasu biblioteki są nieutrzymywane i nie współpracują z nowszymi wersjami pythona. Za pomocą komendy `source uavproject/venv/bin/activate` uruchamiane jest wirtualne środowisko. Wewnątrz środowiska można testować napisane programy i instalować wymagane biblioteki. Pisanie programu najlepiej zacząć używając szablonu 3.1.

Listing 3.1: Szablon programu działającego na komputerze asystującym

```
1 from dronekit import connect
2 from pymavlink import mavutil
3 import os
4 import time
5
6 DRONE_PORT = """/dev/serial/by-id/usb-Holybro_Pixhawk6C_
7             450046001751333337363133-if00""
8
9 def connect_pixhawk():
10     print("Waiting for Pixhawk heartbeat...")
11
12     while True:
13         try:
14             master = mavutil.mavlink_connection(
15                 DRONE_PORT,
16                 baud=115200
17             )
```

```
18
19         master.wait_heartbeat(timeout=60)
20         print("Pixhawk heartbeat received")
21
22         vehicle = connect(
23             DRONE_PORT,
24             baud=115200,
25             wait_ready=False
26         )
27
28         print("Connected to Pixhawk")
29         return vehicle
30
31     except Exception as e:
32         print(f"Pixhawk not ready: {e}")
33         time.sleep(3)
34 def main():
35     vehicle = None
36
37     while not os.path.exists(DRONE_PORT):
38         time.sleep(1)
39     try:
40         vehicle = connect_pixhawk()
41
42         vehicle.message_factory.statustext_send(
43             mavutil.mavlink.MAV_SEVERITY_ALERT,
44             b"Companion computer connected"
45         )
46         vehicle.flush()
47
48         #miejsce na kod
49
50     except KeyboardInterrupt:
51         print("\nProgram interrupted by user.")
52
53     finally:
54         print("Closing connections...")
55
56         if vehicle:
57             vehicle.close()
58             print("Drone connection closed")
59
60 if __name__ == "__main__":
61     main()
62
```

Najważniejszym jego elementem jest funkcja `connect_pixhawk()` otwierająca połączenie z komputerem pokładowym. Pierwszym etapem jest użycie biblioteki PyMAVLink do nasłuchiwania sygnału *heartbeat*. Sygnał ten jest wysyłany przez komputer pokładowy w momencie gdy ten jest uruchomiony i gotowy do pracy. Po otrzymaniu sygnału tworzone jest drugie połączenie z użyciem biblioteki DroneKit, które jest zwracane i może być użyte do komunikacji z dronem w dalszej części programu. Program, używając otwartego połączenia, wywołuje wysłanie wiadomości przez komputer Pixhawk o pomyślnym podłączeniu komputera asystującego. Wiadomość może być odczytana przez pilota mającego połączenie z jednostką dzięki aplikacji Mission Planner. Szablon uwzględnia również poprawne zamknięcie połączenia, również w przypadku jeśli działanie programu zakończy się poprzez przerwanie skrótem klawiszowym lub błąd programu. Bez zamknięcia połączenia każda kolejna próba uruchomienia programu zakończy się błędem, ponieważ urządzenie bez poprawnie zamkniętego połączenia będzie oznaczone jako zajęte i niedostępne.

Loty dronem najczęściej nie odbywają się w warunkach domowych, dlatego programy powinny się wykonywać automatycznie, bez potrzeby użycia klawiatury i monitora. W tym celu wykorzystany został skrypt 3.2 zapisany w `/etc/systemd/system/autorunuav.service`. Po uruchomieniu Raspberry Pi skrypt poczeka aż system będzie w pełni uruchomiony, po czym wykona program określony w parametrze `ExecStart` przy użyciu wirtualnego środowiska. W przypadku jeśli program przestanie działać skrypt odczeka pięć sekund i uruchomi go ponownie. Po każdej zmianie automatycznie uruchamianego programu lub jakichkolwiek innych zmian w skrypcie należy odświeżyć systemd komendą `sudo systemctl daemon-reload` oraz uruchomić skrypt za pomocą `sudo systemctl start autorunuav.service`. Wyjście programu można zobaczyć używając komendy `journalctl -u autorunuav.service -f`.

Listing 3.2: Skrypt wykonujący program po uruchomieniu komputera asystującego

```
1 [Unit]
2 Description=UAV Companion Autostart
3 After=multi-user.target
4
5 [Service]
6 Type=simple
7 User=uavcompanion
8 Group=uavcompanion
9
10 WorkingDirectory=/home/uavcompanion/uavproject
11 ExecStart=/home/uavcompanion/uavproject/venv/bin/python -u program.py
12
```

```
13 Restart=on-failure
14 RestartSec=5
15
16 StandardOutput=journal
17 StandardError=journal
18
19 [Install]
20 WantedBy=multi-user.target
21
```

3.2 Przykład użycia

W celu przetestowania zbudowanej platformy i zaprezentowania jednej z możliwości jej wykorzystania zbudowano system ostrzegania przed kolizją. Wykrywanie przeszkód odbywa się z użyciem trzech ultradźwiękowych czujników odległości HC-SR04 [9]. Zasilanie 5V oraz uziemienie są połączone równolegle do pinów w Raspberry Pi. Piny TRIG w czujnikach, wywołujące pomiar, są podłączone bezpośrednio do pinów GPIO. Wyjście ECHO czujników nie może być podłączone do Raspberry Pi ze względu na różnicę w napięciach. W czujnikach HC-SR04 dla sygnału wychodzącego przez pin ECHO stan wysoki to 5V. Takie napięcie jest odpowiednie dla mikrokomputera Arduino, jednak stan wysoki dla pinów w Raspberry Pi to 3,3V. Aby zapobiec uszkodzeniom należy obniżyć napięcie wychodzące z czujnika do bezpiecznych 3,3V, co zostało osiągnięte z użyciem konwertera poziomów logicznych. Zaprojektowane i wydrukowane na drukarce 3D uchwyty pozwalają na zamontowanie czujników i konwertera na dronie oraz regulację ich pozycji w poziomie. Program 3.3 wywołuje wysłanie ostrzeżenia do pilota w przypadku jeśli wykryta zostanie przeszkoda oraz dron zbliża się do niej z odpowiednią prędkością. Zmienne globalne na początku programu określają między innymi minimalną odległość i prędkość przy których program powinien wywołać ostrzeżenie, wymaga również określenia kątów pod jakimi ustawione są czujniki. Wartość 0 oznacza czujnik zwrócony na wprost, wartości ujemne oznaczają odchylenie w lewo a wartości dodatnie oznaczają odchylenie w prawo. Dla wygody użytkownika kąty wyrażone są w stopniach. Po ustanowieniu połączenia z komputerem pokładowym program w nieskończonej pętli odczytuje odległości z czujników, zostawiając przerwę między odczytami równą wartości określonej w zmiennej `GAP`. W przypadku użycia wielu czujników obok siebie istnieje ryzyko że po wywołaniu pomiaru u jednego z czujników, przejdziemy do drugiego zanim poprzedni zakończy pomiar. W takiej sytuacji może się zdarzyć że fala ultradźwiękowa wysłana przez poprzedni czujnik dotrze do kolejnego, co skutkować będzie zaburzonym pomiarem i fałszywie krótkim zmierzonym dystan-

sem. Ryzyko wystąpienia takiego problemu jest największe gdy czujniki zwrócone są w tę samą stronę, w przypadku zbudowanego systemu ostrzegania ryzyko zaburzenia pomiaru z tego powodu jest znikome. Mimo wszystko pozostawienie odstępu jest wskazane, a odstęp 50ms nie jest wystarczająco długi aby uczynić program zawodnym przez zbyt rzadkie pomiary. Każda wartość pomiaru zwracana jest w postaci liczby zmiennoprzecinkowej pomiędzy 0 a 1, gdzie 0 oznacza 0cm a 1 oznacza maksymalną odległość pomiaru określaną w zmiennej `MAX_DIST`. Wartość pomiaru jest mnożona przez 100 i maksymalną odległość pomiaru w celu uzyskania wartości w centymetrach. Komputer Pixhawk dostarcza programowi informacji o prędkości drona i jego odchyleniu od kierunku północnego. Prędkość drona jest wyrażona w postaci trzech wartości: prędkości w kierunku północnym, wschodnim i w dół. Dzięki odchyleniu od kierunku północnego obliczana jest prędkość względem BSP. Program następnie oblicza prędkość względem danego czujnika, wykorzystując określony wcześniej kąt jego odchylenia względem przodu. Mając te dane program porównuje odczytaną z czujników odległość z określonym progiem aktywacji oraz obecną prędkość w kierunku w który skierowany jest czujnik z minimalną prędkością aktywującą ostrzeżenie. Jeśli odległość jest mniejsza a prędkość większa niż punkty aktywacji, program wywoła wysłanie ostrzeżenia.

3.3 Kod programu

Listing 3.3: Program ostrzegający przed kolizją

```

1  import time
2  import math
3  from dronekit import connect
4  from pymavlink import mavutil
5
6  from gpiozero import Device, DistanceSensor
7  from gpiozero.pins.lgpio import LGPIOWFactory
8
9  Device.pin_factory = LGPIOWFactory()
10
11 DRONE_PORT = "/dev/serial/by-id/usb-Holybro_Pixhawk6C_450046001751333337363133-if0
12
13 MIN_DISTANCE_CM = 30
14 MIN_SPEED = 0.3 # w m/s
15 GAP = 0.05 # 50ms
16 MAX_DIST = 8
17
18 SENSOR_ANGLES_DEG = {
19     1: -45.0,
20     2: 0.0,
```

```
21     3: 45.0
22 }
23
24 sensor1 = DistanceSensor(echo=17, trigger=23, max_distance=MAX_DIST)
25 sensor2 = DistanceSensor(echo=27, trigger=24, max_distance=MAX_DIST)
26 sensor3 = DistanceSensor(echo=22, trigger=25, max_distance=MAX_DIST)
27
28
29 def connect_pixhawk():
30     while True:
31         try:
32             conn = mavutil.mavlink_connection(
33                 DRONE_PORT,
34                 baud=115200
35             )
36
37             conn.wait_heartbeat(timeout=60)
38             conn.close()
39
40             vehicle = connect(
41                 DRONE_PORT,
42                 baud=115200,
43                 wait_ready=True
44             )
45
46             return vehicle
47
48         except Exception:
49             time.sleep(3)
50
51
52 def get_body_velocity(vehicle):
53     velocity = vehicle.velocity
54     yaw = vehicle.attitude.yaw
55
56     if velocity is None or yaw is None:
57         return 0.0, 0.0
58
59     vn, ve, _ = velocity
60
61     v_forward = vn * math.cos(yaw) + ve * math.sin(yaw)
62     v_right = -vn * math.sin(yaw) + ve * math.cos(yaw)
63
64     return v_forward, v_right
65
```

```
66
67 def get_speed_toward_sensor(v_forward, v_right, sensor_angle_deg):
68     angle_rad = math.radians(sensor_angle_deg)
69     return v_forward * math.cos(angle_rad) + v_right * math.sin(angle_rad)
70
71
72 def send_alert(vehicle):
73     message = "obstacle"
74
75     vehicle.message_factory.statustext_send(
76         mavutil.mavlink.MAV_SEVERITY_ALERT,
77         message.encode("utf-8")
78     )
79     vehicle.flush()
80
81
82 def check_distance(vehicle, distance_cm, sensor_id, v_forward, v_right):
83     sensor_angle = SENSOR_ANGLES_DEG[sensor_id]
84     toward_speed = get_speed_toward_sensor(v_forward, v_right, sensor_angle)
85
86     if distance_cm < MIN_DISTANCE_CM and toward_speed > MIN_SPEED:
87         send_alert(vehicle)
88
89
90 def main():
91     vehicle = None
92
93     try:
94         vehicle = connect_pixhawk()
95
96         vehicle.message_factory.statustext_send(
97             mavutil.mavlink.MAV_SEVERITY_ALERT,
98             b"Companion computer connected"
99         )
100     vehicle.flush()
101
102     while True:
103         v_forward, v_right = get_body_velocity(vehicle)
104
105         d1 = sensor1.distance * 100 * MAX_DIST
106         check_distance(vehicle, d1, 1, v_forward, v_right)
107         time.sleep(GAP)
108
109         d2 = sensor2.distance * 100 * MAX_DIST
110         check_distance(vehicle, d2, 2, v_forward, v_right)
```

```
111         time.sleep(GAP)
112
113         d3 = sensor3.distance * 100 * MAX_DIST
114         check_distance(vehicle, d3, 3, v_forward, v_right)
115         time.sleep(GAP)
116
117     except KeyboardInterrupt:
118         pass
119
120     finally:
121         if vehicle:
122             vehicle.close()
123
124
125 if __name__ == "__main__":
126     main()
127
```

3.4 Testy

3.4.1 Sprawdzenie poprawności działania systemu

W celu weryfikacji działania zbudowanego systemu wykonano szereg testów w trakcie rozwoju, które potem powtórzono po ukończeniu prac. Czujniki odległości zostały przetestowane poprzez ustawienie płaskiej przeszkody prostopadle do czujnika, w odległościach 30cm, 50cm, 1m, 2m i 4m. We wszystkich przypadkach wykonane pomiary nie odbiegały od rzeczywistej odległości powyżej 1cm różnicy, co jest wystarczającą dokładnością dla wykrywania przeszkód. W celu sprawdzenia działania programu biorącego pod uwagę prędkość zbliżania się do przeszkody wykonano imitację przeszkody montowaną na BSP kilka centymetrów przed jednym z czujników odległości. Dron następnie był wznoszony w powietrze i rozpędzany do prędkości przekraczającej minimalną wymaganą do wysłania ostrzeżenia. Próbę powtórzono dla wszystkich trzech czujników.

Połączenie między komputerem pokładowym i asystującym jest weryfikowane poprzez wysłanie komunikatu o pomyślnym otwarciu połączenia. Komunikat ten może być odczytany przy użyciu komputera z aplikacją Mission Planner.

3.4.2 Pobór prądu

Pobór prądu Raspberry Pi w trakcie działania programu został zmierzony z użyciem miernika KWS-2302C. Pomiary wahały się pomiędzy 3,371W a 3,612W, w więk-

szości oscylując około 3,491W. Biorąc pod uwagę parametry akumulatora zasilającego komputer asystujący można oszacować czas pracy na jednym ładowaniu na około 10,5 godziny.

3.4.3 Dokładność pomiarów odległości

3.4.4 Czas reakcji systemu

3.5 Alternatywne rozwiązania

W trakcie pracy nad systemem zapobiegającym kolizjom powstały alternatywne rozwiązania i pomysły które nie trafiły do końcowej wersji. Przykładowo program biorący pod uwagę prędkość drona przy decydowaniu o wysłaniu ostrzeżenia został napisany na podstawie wersji sprawdzającej jedynie odległość od przeszkody. W zależności od preferencji użytkownika uzależnienie ostrzeżeń od prędkości może być niepożądane.

Problem z różnicą napięć między stanem wysokim dla czujników odległości a stanem wysokim dla pinów GPIO w Raspberry Pi może być rozwiązany na inne sposoby niż z życiem konwertera stanów logicznych. Istnieje możliwość stworzenia dzielnika napięcia z pomocą dwóch odpowiednio dobranych rezystorów. Z powodu braku takich pod ręką ta możliwość nie została jednak przetestowana. Podczas prac, aby przyspieszyć budowę platformy i przejść do oprogramowania, czujniki odległości zostały podłączone do Arduino Nano. Wywoływanie pomiarów i obliczanie na ich podstawie zmierzonej odległości odbywało się na mikrokontrolerze, a wyniki przesyłane były do Raspberry Pi dzięki połączeniu USB. Takie rozwiązanie wprowadza jednak niepotrzebnie dodatkowe urządzenie do układu oraz zwiększa jego zużycie energii przez konieczność zasilenia mikrokontrolera. W przypadku gdy Raspberry Pi będzie działać z niedostatecznym zasilaniem może ograniczyć prąd idący do urządzeń połączonych przez USB do 600mA [6], w przypadku Arduino Nano takie ograniczenie nie stanowi jednak problemu ponieważ prąd wymagany do jego działania jest znacznie niższy. Użycie Raspberry Pi oraz Arduino komplikuje pracę nad oprogramowaniem; Do działającego systemu trzeba napisania dwóch programów w dwóch różnych językach, oddzielnie dla obu z urządzeń. Z powodu wymienionych wad użycia Arduino zostało ono zastąpione konwerterem stanów logicznych w późniejszych etapach rozwoju, pozostaje on jednak możliwym rozwiązaniem.

Użyte w tym przypadku ultradźwiękowe czujniki odległości nie są idealnymi urządzeniami pomiarowymi. Wykorzystanie fali dźwiękowej odbijającej się od przeszkody do określenia odległości wiąże się z ograniczeniami i podatnościami. Fala dźwiękowa, padająca na przeszkodę, odbija się różnie w zależności od jej materiału i kształtu. Do-

brze wygłuszający materiał taki jak grube płótno czy gąbka może zostać niewykryty przez czujnik ultradźwiękowy. Czujniki wykorzystujące ultradźwięki mają również problem z wykrywaniem przeszkód ustawionych pod zbyt dużym kątem. Idealne warunki pomiarowe to przeszkoda ustawiona prostopadle do czujnika, dla HC-SR04 dopuszczalne odchylenie to 15° [9]. Naukowcy Manabu Ishihara, Makoto Shiina i Shin-nosuke Suzuki przeprowadzili szereg eksperymentów z użyciem czujnika ultradźwiękowego [7] i odkryli że odległość od przeszkody ma wpływ na dopuszczalny kąt pomiaru. Wraz ze skracaniem dystansu można pozwolić sobie na coraz większy kąt i zachowanie niezawodności czujnika. Alternatywą dla czujników ultradźwiękowych mogą być moduły oparte o lasery podczerwone. Ponieważ wiązka lasera porusza się z prędkością światła, znacznie szybciej niż prędkość fali dźwiękowej, pomiary laserem są o wiele szybsze od tych wykonywanych przez czujnik ultradźwiękowy. Używanie wielu czujników laserowych nie wiąże się też z ryzykiem zaburzania odczytów przez siebie nawzajem, co w praktyce eliminuje potrzebę ustawiania opóźnień między pomiarami. Z drugiej strony użycie lasera sprawia że obiekty szklane lub odbijające światło nie zostaną poprawnie wykryte. Czujniki wykorzystujące laser podczerwony są też zazwyczaj droższe niż czujniki ultradźwiękowe. Zastosowanie czujników HC-SR04 motywowane było głównie ich niską ceną oraz dostępnością, jednak wymienione wcześniej zalety czujników laserowych czynią z nich wartą przemyślenia alternatywę.

Rozdział 4

Potencjalne ścieżki rozwoju projektu

- 4.1 Półautonomiczny system zapobiegania kolizjom
- 4.2 Wielokierunkowy monitoring
- 4.3 Sieć neuronowa rozpoznawanie obiektów z obrazu kamery
- 4.4 mapa 3d lidar

Bibliografia

- [1] ArduPilot. Mission planner overview, 2026. dostę: 13.02.2026.
- [2] EASA. Drone regulatory system, 2026. dostę: 11.02.2026.
- [3] Holybro. Air2216ii, blheli s 20a, t1045ii documentation. Technical report, 2026. dostę: 13.02.2026.
- [4] Holybro. M10 gps module, 2026. dostę: 13.02.2026.
- [5] Raspberry Pi Ltd. Raspberry pi 5 release article, 2026. dostę: 27.04.2026.
- [6] Raspberry Pi Ltd. Raspberry pi documentation, 2026. dostę: 26.04.2026.
- [7] Shin-nosuke Suzuki Manabu Ishihara, Makoto Shiina. Evaluation of method of measuring distance between object and walls using ultrasonic sensors. *Journal of Asian Electric Vehicles*, 7(1):1207–1211, June 2009.
- [8] PX4. Pixhawk 6c overview, 2026. dostę: 13.02.2026.
- [9] Cytron Technologies. Hc-sr04 user manual, 2013. dostę: 11.05.2026.